

SIGIC — Documentação Complementar

Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia — Aurora Siger

Gabriel Carmona Bittencourt — RM569239

Marcio Francisco dos Santos Junior — RM570758 Íuri Leão de Almeida — RM570215

Maria Sophia Domingues dos Santos — RM571209

Sumário

1. Visão Geral do Projeto	2
1.1 Estrutura de arquivos	2
2. Organização da Infraestrutura (item 1.1)	3
3. Representação da Rede com Grafos (item 1.2)	4
3.1 Justificativa da estrutura da rede	4
3.2 Dupla representação: lista E matriz de adjacência	4
4. Algoritmos de Rede Implementados (item 1.3)	5
4.1 Dijkstra (caminho mínimo)	5
4.2 Detecção de pontos críticos	5
5. Estruturas de Dados em Python (item 1.4)	6
5.1 Listas	6
5.2 Matrizes (listas de listas)	6
5.3 Tuplas	6
5.4 Dicionários	6
6. Modelagem Matemática e Otimização (item 1.5)	7
6.1 Fórmula utilizada	7
6.2 Explicação das variáveis	7
6.3 Análise qualitativa da função	7
6.4 Relação com o funcionamento da colônia	7
6.5 Otimização	7
6.6 Modelo secundário — perda energética por distância	8
7. Sustentabilidade e Governança — Reflexão ESG (item 1.6)	9
8. Funcionalidades do Menu (item 2.2)	11

1. Visão Geral do Projeto

O **SIGIC (Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia)** representa computacionalmente a infraestrutura da base marciana *Aurora Siger* como um **grafo não direcionado e ponderado**, e oferece um conjunto de algoritmos para otimizar o funcionamento da rede energética e operacional.

O sistema é executado integralmente em **Python puro** (biblioteca padrão apenas — `heapq`, `collections`, `math`, `os`, `sys`), sem nenhuma dependência externa, conforme a regra técnica do item 2.4 do enunciado.

Indicador	Valor
Módulos (vértices)	10
Conexões (arestas)	20
Algoritmos de grafo	BFS, DFS, Dijkstra, pontos de articulação
Modelo matemático	Crescimento exponencial do consumo + cálculo diferencial

1.1 Estrutura de arquivos

```
codigo_fonte.py          # Arquivo principal de execução
modules/
  module.py              # Classe Module (um módulo da colônia)
  grafo.py               # Classe InfrastructureGraph (o grafo)
algorithms/
  dijkstra.py            # Algoritmo de Dijkstra (caminho mínimo)
  analysis.py            # BFS, DFS, pontos críticos, centralidade, eficiência
modeling/
  math.py                # Modelagem matemática e cálculo diferencial
data/
  data_modules.py        # Dados padronizados dos módulos e conexões
ui/
  menu.py                # Menu interativo de terminal
visualization/
  network_viz.py         # Geração de descrições textuais da rede
arquivos_auxiliares/
  gerar_rede_pdf.py      # Gerador do diagrama rede_colonia.pdf (Graphviz)
```

Execução: `python3 codigo_fonte.py`

2. Organização da Infraestrutura (item 1.1)

Cada módulo é uma instância da classe `Module` (`modules/module.py`) e carrega os atributos exigidos pelo enunciado. Os dados padronizados estão em `data/data_modules.py`.

ID	Módulo	Consumo (kWh)	Prioridade	Capacidade (kWh)	Comunicação
CTR-01	Centro de Controle	120	10	15	10
OXI-01	Produção de Oxigênio	100	10	40	4
HAB-01	Habitação	85	9	30	5
MED-01	Suporte Médico	65	9	8	7
ARM-01	Armazenamento de Energia	45	8	500	3
COM-01	Comunicação	75	8	5	10
AGR-01	Agricultura	95	7	20	6
LAB-01	Laboratório Científico	110	6	10	8
MAN-01	Oficina de Manutenção	50	5	5	4
REC-01	Centro de Recreação	40	4	2	3

Os atributos modelam diretamente os requisitos do item 1.1: **consumo energético**, **prioridade operacional**, **capacidade de armazenamento**, **necessidade de comunicação** e **status operacional** (`active`, `maintenance`, `alert`). A **distância entre módulos** é representada como peso das arestas (seção 3).

3. Representação da Rede com Grafos (item 1.2)

A rede é modelada por um **grafo não direcionado e ponderado** na classe `InfrastructureGraph` (`modules/grafos.py`):

- **Vértices** → módulos da colônia;
- **Arestas** → conexões físicas/lógicas entre módulos;
- **Pesos** → distância/custo de transmissão entre os módulos (quanto maior, mais “caro” é enviar energia ou dados por aquela conexão).

O diagrama visual da rede está no arquivo **rede_colonia.pdf**, gerado a partir dos dados reais pelo script `arquivos_auxiliares/gerar_rede_pdf.py` (Graphviz). Nele, a cor do nó indica a prioridade e a cor da aresta indica o tipo de conexão (energia, dados ou suporte à vida).

3.1 Justificativa da estrutura da rede

A topologia foi desenhada com três princípios:

1. **Centralidade dos módulos críticos.** O *Centro de Controle* (CTR-01) e o *Armazenamento de Energia* (ARM-01) são os nós de maior grau, funcionando como *hubs* — refletem a realidade de que controle e energia precisam alcançar toda a base.
2. **Redundância em torno do suporte à vida.** *Produção de Oxigênio* (OXI-01) e *Suporte Médico* (MED-01) possuem mais de um caminho de acesso, reduzindo o risco de isolamento em caso de falha.
3. **Pesos coerentes com a função.** Conexões vitais e curtas (ex.: Habitação—Suporte Médico, peso 1) têm peso baixo; conexões de apoio mais distantes têm peso maior.

3.2 Dupla representação: lista E matriz de adjacência

O grafo mantém **simultaneamente** as duas representações clássicas, cada uma usada onde é mais eficiente:

Representação	Atributo	Usada para
Lista de adjacência	<code>adjacency_list</code> (dict de listas)	Percorrer vizinhos em BFS/DFS/Dijkstra — eficiente em grafos esparsos
Matriz de adjacência	<code>adjacency_matrix</code> (lista de listas)	Consulta direta da relação entre dois módulos em $O(1)$
Matriz de distâncias	<code>distance_matrix</code>	Referência rápida de pesos entre pares

4. Algoritmos de Rede Implementados (item 1.3)

Algoritmo	Função	Objetivo (enunciado)
BFS	<code>SearchAlgorithms.bfs</code>	Explorar a rede por níveis
DFS	<code>SearchAlgorithms.dfs</code>	Explorar caminhos em profundidade
Dijkstra	<code>Dijkstra.find_path</code>	Caminho mínimo / rota mais eficiente
Dijkstra com restrição	<code>find_path_with_constraints</code>	Otimizar rota respeitando prioridade
Pontos de articulação	<code>detect_critical_points</code>	Detectar pontos críticos da rede
Centralidade	<code>analyze centrality</code>	Identificar <i>hubs</i> da rede
Eficiência	<code>analyze_efficiency</code>	Avaliar desempenho operacional

Algoritmos de busca e análise em `algorithms/analysis.py`; Dijkstra em `algorithms/dijkstra.py`.

4.1 Dijkstra (caminho mínimo)

Implementado com **fila de prioridade** (`heapq`) para complexidade $O((V + E) \log V)$. Responde ao exemplo do enunciado: dado o *Armazenamento de Energia* como origem e o *Suporte Médico* como destino, retorna a rota de menor custo total e a distância acumulada.

4.2 Detecção de pontos críticos

Usa o algoritmo de **pontos de articulação** (baseado em DFS, com tempos de descoberta e valores *low* — abordagem de Tarjan). Um módulo é crítico quando sua remoção desconecta a rede, indicando onde a infraestrutura é vulnerável e precisa de redundância.

5. Estruturas de Dados em Python (item 1.4)

As quatro estruturas exigidas são usadas de forma deliberada, cada uma onde suas propriedades são vantajosas:

5.1 Listas

- `module_list`: ordem de iteração estável dos módulos;
- valores de `adjacency_list`: lista de vizinhos de cada vértice;
- caminhos retornados por BFS/DFS/Dijkstra (`path`).

Justificativa: sequências mutáveis e ordenadas, ideais para iteração e para construir caminhos passo a passo.

5.2 Matrizes (listas de listas)

- `adjacency_matrix` e `distance_matrix`: relação entre pares de módulos.

Justificativa: acesso indexado $O(1)$ à relação entre dois vértices, complementando a lista de adjacência.

5.3 Tuplas

- `Module._fixed_data`: os 7 atributos **imutáveis** de cada módulo;
- posições (`x`, `y`) em `MODULE_POSITIONS`;
- retorno (`caminho`, `distância`) do Dijkstra.

Justificativa: a imutabilidade protege dados que **não devem mudar** após a criação do módulo (ID, consumo nominal, prioridade), evitando alterações acidentais.

5.4 Dicionários

- `modules` (`id` $\rightarrow$ `Module`): acesso $O(1)$ por identificador;
- `edge_weights` ("`id1-id2`" $\rightarrow$ `peso`);
- `adjacency_list` (`defaultdict(list)`);
- `connection_types` e `_attributes` (atributos dinâmicos do módulo).

Justificativa: mapeamento chave $\rightarrow$ valor com busca em tempo constante — essencial para a performance dos algoritmos que consultam módulos e pesos repetidamente.

6. Modelagem Matemática e Otimização (item 1.5)

Arquivo: `modeling/math.py`. O fenômeno principal modelado é o **crescimento do consumo energético** da colônia ao longo do tempo, com aplicação de **cálculo diferencial**.

6.1 Fórmula utilizada

$$C(t) = C_0 \cdot e^{r \cdot t}$$

6.2 Explicação das variáveis

Símbolo	Significado
$C(t)$	Consumo energético total no instante t (kWh)
C_0	Consumo inicial — soma do consumo de todos os módulos (785 kWh)
r	Taxa de crescimento anual (padrão $0,12 = 12\%$ ao ano)
t	Tempo, em anos

A **derivada primeira** $C'(t)$ (calculada numericamente por diferença central) mede a **velocidade** de crescimento do consumo; a **derivada segunda** $C''(t)$ mede sua **aceleração** (concavidade da curva).

$$C'(t) \approx \frac{C(t+h) - C(t-h)}{2h} \quad C''(t) \approx \frac{C(t+h) - 2C(t) + C(t-h)}{h^2}$$

6.3 Análise qualitativa da função

Por ser uma **exponencial com** $r > 0$, a função é **monotonicamente crescente** e **convexa**: tanto $C'(t)$ quanto $C''(t)$ são positivas para todo t . Ou seja, o consumo não apenas cresce, mas cresce **cada vez mais rápido**. Não há ponto de máximo ou mínimo interno — o mínimo ocorre em $t = 0$. Isso é interpretado pelo sistema em `MathematicalModeling._interpret_rate_of_change` como “*crescimento acelerado do consumo (curva convexa)*”.

6.4 Relação com o funcionamento da colônia

O modelo permite **prever quando a demanda energética se aproximará da capacidade total** da base (`predict_critical_point`) e simular **cenários** otimista (8%), moderado (12%) e pessimista (18%) de crescimento (`simulate_scenarios`). Esses resultados orientam decisões de **expansão e investimento em eficiência** antes que a colônia atinja um ponto crítico.

6.5 Otimização

- `optimize_energy_distribution`: usa derivadas (método de Newton numérico) para buscar o ponto de **máxima eficiência** de distribuição por módulo.
- `optimal_consumption_point`: aplica o **teste da segunda derivada** para classificar pontos críticos (mínimo, máximo ou inflexão).

6.6 Modelo secundário — perda energética por distância

$$P_{\text{perda}}(d) = 1 - e^{-d \cdot (1-\eta)}$$

onde d é a distância (peso da aresta) e η a eficiência de transmissão (95%). Modela o fato de que conexões mais longas perdem mais energia, seguindo uma curva exponencial.

7. Sustentabilidade e Governança — Reflexão ESG (item 1.6)

Na Terra, os critérios ESG (ambiental, social e governança) nasceram como uma forma de cobrar das organizações um equilíbrio entre resultado econômico, impacto sobre o planeta e responsabilidade para com as pessoas. Em uma colônia marciana, esses mesmos critérios deixam de ser um compromisso voluntário e tornam-se a própria condição de sobrevivência. Marte não oferece externalidades: não há atmosfera para diluir desperdícios, não há ecossistema para absorver erros e não há resgate a poucas horas de distância. A Aurora Siger é um sistema fechado no qual cada decisão técnica é, simultaneamente, uma decisão ética. O SIGIC foi concebido sob essa premissa — traduzir princípios ESG em escolhas computacionais objetivas e auditáveis.

Dimensão ambiental. Em um ambiente sem biosfera, “sustentabilidade ambiental” significa, antes de tudo, a gestão implacável da energia — o recurso-mestre do qual todos os demais dependem, já que oxigênio, água, calor e alimento são, em última análise, energia transformada. O modelo de crescimento exponencial do consumo, $C(t) = C_0 e^{r \cdot t}$, funciona como um alerta: mesmo taxas modestas de expansão conduzem, no longo prazo, a uma demanda capaz de superar a capacidade instalada. O **uso sustentável de energia**, portanto, não é um ideal abstrato, mas uma restrição física inescapável. A modelagem de perda energética por distância mostra que cada conexão longa dissipa, de forma irrecuperável, parte da energia transmitida; assim, a **redução de desperdícios** equivale, literalmente, à recuperação de margem de vida. Minimizar perdas, priorizar fontes renováveis e encurtar rotas de distribuição não são gestos de marketing verde, mas medidas que ampliam o tempo de autonomia da base. Mais do que isso, a lógica circular impõe-se naturalmente: em um sistema fechado, todo rejeito é insumo em potencial, e a sustentabilidade confunde-se com a capacidade de reaproveitar calor, água e materiais que, na Terra, seriam simplesmente descartados.

Dimensão social. O pilar social do ESG, na Terra associado a condições de trabalho e impacto comunitário, assume na colônia a forma mais radical possível: a manutenção da própria vida da tripulação. Os módulos de Produção de Oxigênio, Suporte Médico e Habitação concentram as maiores prioridades operacionais precisamente porque sua falha é incompatível com a sobrevivência humana. A **priorização de sistemas críticos** é, nesse sentido, uma decisão moral codificada em software: quando a energia é escassa, o sistema deve assegurar que ela flua primeiro para onde a vida está em risco. O algoritmo de Dijkstra com restrição de prioridade materializa esse princípio, recusando rotas que sacrificariam módulos essenciais em nome de uma eficiência puramente numérica. A comunicação constante — entre módulos e com a Terra — também é um bem social: o isolamento extremo transforma o fluxo de informação em fator de saúde psicológica e de coesão da equipe, não em luxo dispensável. Sob escassez, emerge ainda uma questão de justiça distributiva: definir quem recebe energia, e em que ordem, é estabelecer na prática uma hierarquia de necessidades que precisa ser legítima aos olhos de toda a tripulação.

Dimensão de governança. Em condições de isolamento, a legitimidade das decisões depende de sua transparência. Uma escolha arbitrária ou opaca sobre quem recebe energia, ou sobre quando desligar um módulo, corrói a confiança da tripulação e, no limite, pode custar vidas. A **governança tecnológica** da Aurora Siger sustenta-se sobre critérios mensuráveis: as métricas de eficiência, de centralidade e os pontos de articulação da rede oferecem uma base racional e reproduzível para a tomada de decisão, no lugar de julgamentos improvisados. É essencial, contudo, compreender o SIGIC como um sistema de apoio: ele recomenda, projeta cenários e expõe vulnerabilidades, mas a responsabilidade última permanece humana. Essa fronteira — algoritmos que informam, pessoas que decidem — é o próprio cerne de uma governança responsável sobre sistemas automatizados.

A governança também se manifesta no planejamento. A **expansão organizada da colônia** exige antecipar o crescimento da demanda antes que ele se converta em crise. As projeções matemáticas

do sistema permitem dimensionar quando novos módulos serão necessários e qual será seu impacto energético, evitando que a base cresça de modo desordenado e ultrapasse seus próprios limites de sustentação. Crescer sem planejamento, em Marte, não é mera ineficiência: é risco existencial.

Vistos em conjunto, os três pilares revelam uma característica peculiar do ESG marciano: sua indivisibilidade. Na Terra, é possível — ainda que questionável — tratar ambiente, sociedade e governança como agendas separadas. Na Aurora Siger, uma perda energética ambiental converte-se de imediato em risco social (falta de oxigênio) e exige resposta de governança (decidir prioridades). O SIGIC torna essa interdependência visível e operável, transformando conceitos que na Terra soam corporativos em instrumentos concretos de sobrevivência coletiva. Sustentabilidade, aqui, não é um relatório anual: é o sistema operacional de uma civilização em miniatura empenhada em permanecer viva.

8. Funcionalidades do Menu (item 2.2)

O sistema oferece um **menu de terminal** com navegação simples:

1. Visualizar Rede da Colônia
2. Consultar Módulo
3. Algoritmos de Rede (BFS, DFS, Dijkstra, eficiência, pontos críticos, centralidade)
4. Modelagem Matemática (projeção, perdas, derivadas, otimização, cenários)
5. Sustentabilidade e Governança (ESG)
6. Simulações Operacionais (falha, pico de consumo, expansão, otimização)
7. Análise Completa
8. Sobre o Sistema
9. Sair

Mensagens de entrada e saída são compreensíveis, com tratamento de entradas inválidas e exemplos práticos de execução em todas as funcionalidades.